Module 7 — Logging, Monitoring & Observability for AI APIs

Learning Objectives

By the end of this module, you will be able to:

- Implement **standard Python logging** within a FastAPI service to track model execution.
- Distinguish between **debugging** (inspecting a single failure) and **monitoring** (tracking system health over time).
- Utilize **Middleware** to automatically log request metadata and inference latency.
- Capture and log **AI-specific metrics** such as prediction confidence scores and feature distributions.
- Use logs to drive **dataset improvements** and informed model retraining decisions.

Observability Concepts

1. Why AI Systems Need Observability

In traditional software, a crash is obvious. In AI Engineering, models often **fail silently**. A model might return a valid-looking prediction (Status 200 OK) while its mathematical accuracy has plummeted due to a shift in input data. Observability allows you to look inside the "black box" of the model during production.

- **Debugging vs. Monitoring:** Debugging is the act of using tools like the **VS Code Debugger** and breakpoints to find out why one specific request failed. Monitoring is the high-level observation of your API's performance, such as average **latency** or **error rates**, across thousands of users.

2. The AI Logging Strategy: What to Capture

For an AI API, logging isn't just about "errors." It is about capturing the context of the prediction.

- **Request Metadata:** Log the version of the model being used and the timestamp of the request.
- **Prediction Summaries:** Instead of logging raw high-dimensional vectors (which bloat logs), log the final **Label** and **Confidence Score**.
- **Inference Latency:** Capture the time it takes for the model to process the data, as machine learning models are computationally expensive and can slow down the entire system.

Code Examples with Line-by-Line Explanations

Example 1: Basic Logging and Startup Events

This example sets up a logger that records when the model is successfully loaded into memory.

```
import logging # (1)
from fastapi import FastAPI

# (2) Configure the standard Python logger
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("ai_service")

app = FastAPI()

@app.on_event("startup") # (3)
def load_model():
    # Simulate loading heavy model weights
    logger.info("Starting model loading process...") # (4)
    logger.info("Model weights loaded successfully. Version: 2.1.0")
```

Line-by-Line Explanation:

1. **`import logging`**: Uses the standard Python library, which is fully compatible with FastAPI and Uvicorn.
2. **`logging.basicConfig(level=logging.INFO)`**: Sets the sensitivity of the logs. `INFO` ensures we see standard operational messages in the terminal.
3. **`@app.on_event("startup")`**: Ensures the loading logs only happen once when the server starts, not for every request.
4. **`logger.info(...)`**: Writes a message to the console. This helps an AI Engineer verify the model is ready before traffic arrives.

Example 2: Logging inside the `/predict` Endpoint

This demonstrates how to log prediction details to monitor "silent failures."

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import logging

logger = logging.getLogger("inference")
app = FastAPI()

class PredictRequest(BaseModel):
    feature_val: float

@app.post("/predict")
def predict(data: PredictRequest):
    # (1) Log incoming feature context
    logger.info(f"Inference request received for feature: {data.feature_val}")
```

```
    # (2) Simulated Inference Logic
    prediction = "Low Risk" if data.feature_val < 0.5 else "High Risk"
    confidence = 0.95

    # (3) Log the result for monitoring
    logger.info(f"Prediction: {prediction} | Confidence: {confidence}")

    return {"prediction": prediction, "confidence": confidence}
```

Line-by-Line Explanation:

1. **`logger.info(f"Inference request...")`**: Records the input. If the model starts giving "garbage" results, you can check these logs to see if users are sending unexpected data.
2. **Simulated Logic**: This represents your `model.predict()` step.
3. **`logger.info(f"Prediction: ...")`**: Captures the model's output. If you see a hundred "High Risk" predictions in a row, you can identify a potential issue with the data distribution.

Example 3: Middleware for Latency Logging

Middleware acts as a "wrapper" around your entire API to measure performance.

```
import time
import logging
from fastapi import FastAPI, Request

logger = logging.getLogger("performance")
app = FastAPI()

@app.middleware("http") # (1)
async def log_latency(request: Request, call_next):
    start_time = time.time() # (2)

    response = await call_next(request) # (3)

    process_time = time.time() - start_time # (4)
    logger.info(f"Path: {request.url.path} | Latency: {process_time:.4f}s")

    return response
```

Line-by-Line Explanation:

1. **`@app.middleware("http")`**: Registers a function that runs for every single request sent to the API.
2. **`start_time = time.time()`**: Records exactly when the request reached the server.
3. **`await call_next(request)`**: This is the most important line. It "passes" the request to your `/predict` function and waits for the result.
4. **`process_time = ...`**: Calculates the total time taken. Logging this allows you to detect if a new model version is significantly slower than the old one.

AI-Specific Production Scenarios

- **Sudden Drop in Confidence:** By logging confidence scores, you can detect when the model is "unsure" about its predictions. This usually indicates **Out-of-Distribution** data that the model wasn't trained on.
- **Increased Latency after Update:** Machine learning models are heavy. Using the **Middleware** example above, you can see if a model update causes response times to spike from 100ms to 500ms, potentially overloading your container's memory.
- **Retraining Decisions:** When you log inputs and outputs, you are essentially creating a record of "Real-World Data." AI Engineers use these logs to pick difficult cases to include in the next **retraining cycle** to improve the model.

Common Logging Mistakes AI Engineers Make

- **Logging Raw Data (PII):** Logging sensitive user data (like names or medical IDs) in plain text. Always log identifiers or hashed values instead.
- **Blocking Logs:** Using slow, synchronous logging inside an `async` FastAPI function, which can increase the very latency you are trying to measure.
- **Log Level Confusion:** Using `ERROR` for things that are actually `INFO` (like a 404 "Not Found" result), which triggers false alarms in monitoring systems.

Assignment: Add Logging to a Predictor

1. Take your "House Price Predictor" API from Module 3.
2. Add a logger named `price_model`.
3. In the `/predict` endpoint, log the `square_feet` input and the final `estimated_price` output.
4. Implement the **latency middleware** from Example 3 to track how long the math takes.
5. Run the API and perform 3 different predictions via the `/docs` page, then check your terminal to see the logs.

Mini Project: Observable Inference API

Goal: Build a "Model Health Dashboard" service.

- Create an API with a POST `/classify` endpoint.
- On **Startup**, log the model version and available memory.

- Inside the endpoint, use a `try-except` block. If inference fails, log the specific error with `logger.error` before raising an `HTTPException`.
- Log every prediction's result and its processing time.
- Ensure that if a user sends a `confidence_score` below 0.5, a special "Low Confidence Warning" is logged.

Observability Checklist: Is my AI API production-safe?

- [ ] Does the API log a "Ready" message only after the model is fully loaded in the **Startup Event**?
- [ ] Are **Confidence Scores** being logged for every successful inference?
- [ ] Does the API measure and log the **total time (Latency)** for each request?
- [ ] Are technical model crashes (500 errors) logged with their **full error details** for debugging?
- [ ] Have I confirmed that no sensitive data (passwords/PII) is being written to the logs?
- [ ] Can I see the logs in real-time in my terminal while running **`fastapi dev`**?